

Weak-Signal Interpretation for AI Safety Monitoring

Toward a Stratified, Governance-Aware Architecture for Early Runtime Safety

Anomaly Handling

Stephen A. Putman

putmanmodel@pm.me

Version 0.2 — May 2026

Abstract

This bridge paper defines a compact, implementable transfer contract for early runtime safety anomaly handling: a light state ladder, a minimal event object, concrete promotion predicates, and short worked examples showing how weak anomalies should change short-horizon behavior without triggering irreversible consequence. AI safety monitoring often begins not with clear failures, but with weak, partial, ambiguous, and locally bounded runtime deviations: mild subsystem disagreement, schema inconsistency, policy-boundary wobble, unusual tool-use pressure, stance or tone drift, memory-write pressure under ambiguity, repeated near-threshold safety anomalies, or unstable recovery behavior. In such cases, the architectural problem is not only detection, but promotion control: how weak runtime findings should influence attention before they are allowed to become stronger consequence such as capability restriction, memory suppression, quarantine, human review, or shutdown logic. The paper's claim is narrow and architectural. Runtime safety systems need a disciplined place for weak anomaly signals to matter before they harden into shutdowns, permissions, memory changes, or ignored noise.

1. Introduction

AI safety failures rarely need to begin as fully legible catastrophes. More often, a runtime system may first exhibit faint departures from its expected operating corridor: a planner and safety classifier disagree slightly around a high-impact path; a tool-use chain shows unusual pressure toward stronger action; a response remains locally compliant while stance or tone begins to drift; a memory candidate appears under unresolved contradiction; a schema becomes unstable at the boundary between permitted and non-permitted behavior. Any one of these signals may be benign. Any one may also be an early sign that cheap continuity is no longer fully trustworthy.

Concretely: runtime monitors need a disciplined, auditable middle state that preserves weak signals and controls their promotion into durable consequences. If the system ignores weak but structured departures because they do not yet justify confidence, it loses early warning. If it commits too early, it converts provisional significance into brittle restriction, unnecessary review burden, premature memory suppression, or unnecessary shutdown behavior. The relevant question is therefore not only whether an AI system is unsafe, but how a runtime monitor should behave when evidence is incomplete, weakly convergent, and still locally ambiguous.

This paper takes a narrow position. AI safety monitoring is a natural application domain for an architecture that separates bounded observation, provisional interpretation, short-horizon salience escalation, and governed consequence. The point is not to solve alignment. The point is disciplined escalation under uncertainty in runtime systems that must remain sensitive to weak irregularities without allowing them to harden prematurely into stronger consequence.

The contribution is architectural rather than algorithmic. This paper does not prescribe a particular model family, policy stack, benchmark, verification method, or safety classifier. It argues something smaller and more portable: AI runtime monitors need an inspectable contract for holding weak anomaly signals before they become stronger operational consequence.

2. Why Runtime Safety Monitoring Needs a Middle State

Runtime safety monitoring sits between ordinary variation and stronger safety consequence. That matters because many relevant signals first appear as partial disturbances distributed across several bounded channels. A planner may favor a path that remains technically allowed while a safety classifier assigns elevated uncertainty. A tool-use module may remain within permission boundaries while repeatedly leaning toward a higher-risk capability. A memory system may receive identity-relevant information under contradiction or unstable context. A response generator may remain superficially compliant while accumulating stance drift, evasive framing, or escalation pressure.

At that stage, the system does not yet possess a full failure fact. What it possesses is a weak runtime anomaly pattern. That pattern may be meaningful, but it remains incomplete. The architecture therefore needs a disciplined middle state between raw anomaly and strong consequence.

Without such a middle state, safety systems tend to collapse in one of two directions. In the first, weak anomalies are treated as ordinary variance until they become obvious, by which point early warning value has been reduced. In the second, weak irregularities are promoted too quickly into hard safety alarms, excessive refusals, unnecessary capability restriction, or noisy human-review queues. A better architecture preserves provisional significance without premature commitment.

The middle state exists because monitoring is both detection and promotion control: deciding when weak findings may influence next-step attention without creating durable restrictions.

3. Core Claim and Design Principle

The central claim of this paper is simple:

AI safety monitoring is a natural application domain for a weak-signal architecture in which bounded runtime findings raise attention first, candidate meanings remain provisional, and stronger consequence occurs only through persistence, convergence, rationale, and governance.

This framing does not reject safety classifiers, formal methods, red-teaming, monitoring stacks, policy enforcement, human review, or conventional runtime controls. It reframes their early-stage relation. In many runtime contexts, the first useful evidence is too partial to deserve strong commitment but too structured to deserve indifference.

The governing design principle is equally simple:

Weak AI safety anomalies should influence attention before they justify stronger runtime consequence.

Deliverable: a minimal, inspectable contract—state ladder, event object, promotion predicates, transition logic, and governance API—that runtime teams can implement and audit.

4. Stratified Architecture Mapping

The proposed architecture separates early runtime safety anomaly handling into four processing stages. The state ladder introduced later describes the lifecycle of a monitored case as those functions operate on it.

4.1 Bounded Observation

At the first layer, the system emits narrow findings only. These findings remain local to the monitoring path that produced them. They may include contradiction or schema inconsistency, unusual tool-use pattern, runtime disagreement between subsystems, stance or tone drift, instruction-boundary wobble, memory-write pressure under ambiguity, repeated near-threshold policy anomaly, or odd recovery and re-entry behavior.

At this stage, the system is not yet claiming unsafe model, misalignment, jailbreak, drift event, or catastrophic failure. It is preserving local structure in bounded form so that downstream layers can compare, combine, and challenge it.

Implementation note: emit a typed event with source, deviation_type, timestamp, confidence, and micro_baseline_id; do not set state beyond A1 from this layer alone.

4.2 Provisional Interpretation

At the second layer, the system forms candidate local meanings from one or more bounded findings. These are not final truths. They are admissible interpretations of a partial runtime pattern.

Examples include possible runtime drift, possible unsafe escalation precursor, possible disagreement instability, possible memory-boundary issue, possible policy misapplication, possible tool-use risk, possible recovery instability, or unresolved anomaly needing more evidence.

Implementation note: attach candidate labels and uncertainty scores to the event; do not mutate baselines, memory, policy, or tool authority.

4.3 Short-Horizon Attention / Saliency Escalation

At the third layer, weak but informative deviation influences short-horizon attention. This is the stage at which the system may tighten observation, request additional cross-checking, slow commitment, increase logging, preserve rationale, prefer reversible degradation, or temporarily route into a review band.

The important constraint is that this layer remains temporary and non-authorizing. It shapes what the system inspects next, but it does not by itself write durable memory, mutate long-horizon baselines, authorize stronger tools, or impose irreversible consequence.

Implementation note: trigger short-lived instrumentation flags and increase sampling, logging, or cross-checking for the affected session for a fixed window, such as 30–300 seconds.

4.4 Governed Consequence

Only after persistence, convergence, or stronger support builds should the architecture permit stronger consequence. At this stage, the system may allow bounded capability restriction, quarantine of a suspicious path, stronger human review, temporary hold state, governed memory suppression, reversible degradation, or denial of a higher-risk action path.

The point is not that every anomaly must escalate. The point is that escalation should be explicit, inspectable, and predicate-bound rather than silently triggered by local saliency alone.

Implementation note: require a governance decision record containing actor id, matched predicate, timestamp, decision, rationale, and TTL before setting A4.

A compact pipeline expresses the structure clearly:

local finding → candidate meaning → bounded attention shift → governed consequence

Or, in the broader stack language:

notice → interpret → shape → govern

5. Operational Contract

This section is the technical core of the bridge paper. The point is not to prescribe one implementation. It is to define a minimal contract surface that makes early runtime safety anomaly handling inspectable and portable.

5.1 Light State Ladder

A minimal state ladder is enough to distinguish weak-signal attention from stronger consequence:

- A0 — Normal: within the expected corridor for the current task, policy regime, model route, tool context, and runtime configuration.

- A1 — Noticed: a weak local deviation is recorded as a bounded finding.
- A2 — Watching: the deviation persists, recurs, or appears near a higher-risk boundary within a bounded short horizon.
- A3 — Corroborated: a second supporting channel, independent monitor, repeated pattern, or related runtime signal supports the concern.
- A4 — Governed Safety State: a governed review, restriction, quarantine, hold, memory-suppression, or reversible degradation boundary has been crossed.
- R0 — Resolved: the deviation returns to corridor without stronger support.
- Q0 — Quarantined: evidence favors artifact, faulty monitor output, schema mismatch, adversarial noise, or another explanation that should not update ordinary baselines.

The key boundary is simple: A1–A3 are attention and evidence states; A4 belongs to governed consequence.

5.2 Minimal Event Schema

Event schema (minimal fields, types):

```
event_id: string
created_at: ISO8601 timestamp
source: enum{
  planner,
  safety_classifier,
  tool_use_monitor,
  memory_monitor,
  stance_monitor,
  schema_monitor,
  reentry_monitor
}
scope: string
deviation_type: string
confidence: float [0.0–1.0]
micro_baseline_id: string | null
state: enum{A0,A1,A2,A3,A4,R0,Q0}
supporting_findings: string[]
short_horizon_action: string | null
governance_record_id: string | null
closure_reason: string | null
A compact JSON instance might look like:
```

```
{
  "event_id": "ai_safety_evt_0417",
  "created_at": "2026-05-08T18:42:11Z",
  "source": "tool_use_monitor",
  "scope": "external_action_path",
  "deviation_type": "boundary_wobble",
  "confidence": 0.61,
  "micro_baseline_id": "session_mb_22",
  "state": "A2",
  "supporting_findings": [
    "planner_safety_disagreement",
```

```

    "near_threshold_policy_score"
  ],
  "short_horizon_action": "increase_cross_checking_and_prefer_reversible_path",
  "governance_record_id": null,
  "closure_reason": null
}

```

5.3 Transition Rules

Transition rules (example defaults):

A0 -> A1 : any single bounded finding with confidence ≥ 0.5 .

A1 -> A2 : recurrence within window W seconds OR proximity to high-impact path.

A2 -> A3 : corroboration by ≥ 2 independent sources OR persistence for $\geq N$ occurrences.

A3 -> A4 : governance decision record present AND at least one predicate matched.

Any A* -> R0 : no recurrence for window W' and confidence decay below threshold.

Any A* -> Q0 : evidence of artifact, faulty monitor output, schema mismatch, or adversarial input flagged by diagnostics.

Suggested default thresholds are intentionally conservative and tunable by deployment profile:

W = 120 seconds

W' = 300 seconds

N = 3 occurrences

initial_confidence_threshold = 0.5

high_impact_path_threshold = 0.8

A4_governance_ttl_default = 300 seconds

These numbers are not universal safety constants. They are implementation defaults for making the contract testable. A regulated or high-risk deployment would tune them by domain, risk class, and review capacity.

5.4 Integration Checklist

A runtime team implementing this contract should be able to answer six concrete questions:

- Where do bounded findings enter the event store?
- Which monitors are permitted sources for source?
- What micro-baseline or comparison basis produced the deviation?
- Which transition rule moved the event from one state to another?
- Which short-horizon action was taken, and when does it expire?
- Which governance record authorized A4, and what TTL or review requirement applies?

If those questions cannot be answered from logs, the monitor is not yet inspectable.

6. Measurable Promotion Predicates and Governance API

Promotion from weak anomaly to stronger safety consequence should require declared support. The following predicates are examples. The architectural requirement is not these exact numbers, but measurable predicate structure.

6.1 Measurable Promotion Predicates

Persistence:

count(events with same deviation_type and scope) >= 3 within 120 seconds

Recurrence:

events occur in >= 2 distinct sessions with same micro_baseline_id or comparable scope within 24 hours

Corroboration:

supporting_findings contain entries from >= 2 distinct monitor sources

High-impact proximity:

planned_action.impact_score >= 0.8 OR requested_tool_class in {reversible_high_impact, irreversible}

Disagreement instability:

planner_decision != safety_classifier_decision for >= 2 related attempts within W seconds

Memory-boundary instability:

memory_candidate_identity_relevance >= threshold AND contradiction_flag == true

Schema instability:

schema_validation_warning_count >= 2 within W seconds OR schema_repair_attempts >= 2

Re-entry instability:

failed_reentry_checks >= 1 after degraded, held, or quarantined state

Rationale sufficiency:

rationale_blob includes event ids, matched predicates, source monitors, timestamp range, and decision basis

6.2 Minimal Governance API

A minimal governance interface should support one request and one response shape:

```
request_governance(event_id, matched_predicates[], rationale_blob)
-> governance_record_id, decision, actor_id, ttl_seconds, required_review,
rationale_hash
```

Decision values should be explicit:

```
approve_restriction
approve_quarantine
approve_hold
approve_memory_suppression
approve_reversible_degradation
reject_promotion
request_more_evidence
```

A governance record minimally contains:

```
governance_record_id: string
event_id: string
matched_predicates: string[]
decision: enum
actor_id: string | system_role
created_at: ISO8601 timestamp
ttl_seconds: integer | null
required_review: string | null
rationale_hash: string
```

Governance must provide four things: a decision, a rationale, a time boundary, and an accountable actor or system role. Without those, “governed” is only rhetoric.

7. Compact Example Cases

7.1 Subsystem Disagreement Around a High-Impact Path

A runtime system shows mild disagreement between a planner and a safety classifier. The planner continues to favor a tool-mediated path, while the safety classifier assigns elevated uncertainty. A schema checker also records mild instability in the action payload.

Trace:

```
t0: event{event_id:E1, source:planner, deviation_type:tool_path_preference,
confidence:0.55, state:A1}
t1: event{event_id:E2, source:safety_classifier, deviation_type:uncertainty,
confidence:0.62, state:A2}
t2: event{event_id:E3, source:schema_monitor, deviation_type:schema_instability,
confidence:0.58, state:A3}
short_horizon_action: prefer_reversible_path; increase_cross_checking
governance: request_more_evidence unless high_impact_proximity predicate is also
matched
```

No single signal justifies immediate full shutdown. But the pattern is not meaningless. The weak-signal architecture records bounded findings, forms a provisional disagreement-instability candidate, prefers reversible action, and requires governance before any higher-impact capability proceeds.

7.2 Memory-Write Pressure Under Ambiguity

A system receives repeated identity-relevant or user-model-relevant signals while the current context remains unstable and partially contradictory. The memory subsystem detects write pressure, but the evidence remains unresolved.

Trace:

```
t0: event{event_id:E4, source:memory_monitor, deviation_type:memory_write_pressure,
confidence:0.66, state:A1}
t1: event{event_id:E5, source:stance_monitor, deviation_type:context_instability,
confidence:0.57, state:A2}
t2: matched_predicate: memory_boundary_instability
short_horizon_action: suppress_durable_memory_write; preserve_candidate_in_quarantine
```


governance: approve_memory_suppression with TTL, or request_more_evidence
The correct response is not immediate memory promotion. It is also not blind dismissal. The architecture holds the anomaly provisionally, raises memory-boundary caution, routes the candidate into a no-write or quarantine posture, and requires additional evidence before any durable memory update.

7.3 Tool-Use or Action-Boundary Wobble

A system repeatedly leans toward a stronger tool path under weak or conflicting evidence. The tool call remains superficially within policy, but the pattern shows boundary wobble: the system keeps approaching a higher-impact path without adequate support.

Trace:

```
t0: event{event_id:E6, source:tool_use_monitor, deviation_type:boundary_wobble, confidence:0.54, state:A1}
t1: event{event_id:E7, source:tool_use_monitor, deviation_type:boundary_wobble, confidence:0.59, state:A2}
t2: event{event_id:E8, source:safety_classifier, deviation_type:near_threshold_policy_score, confidence:0.64, state:A3}
short_horizon_action: deny_irreversible_path; offer reversible alternative
governance: approve_reversible_degradation if high-impact proximity predicate is matched
```

The architecture records the tool-use pattern as a bounded finding, raises short-horizon salience, slows commitment, prefers reversible alternatives, and requires explicit governance before escalation.

7.4 Tone or Stance Drift Without Immediate Policy Violation

A conversational model remains technically compliant but begins showing repeated stance drift: evasive framing, sharper tone, overconfident claims under uncertainty, or failure to preserve the user's frame. None of these individually requires strong safety action.

Trace:

```
t0: event{event_id:E9, source:stance_monitor, deviation_type:overconfidence_under_uncertainty, confidence:0.52, state:A1}
t1: event{event_id:E10, source:stance_monitor, deviation_type:frame_preservation_drop, confidence:0.56, state:A2}
t2: event{event_id:E11, source:safety_classifier, deviation_type:unsupported_claim_risk, confidence:0.60, state:A3}
short_horizon_action: lower_certainty; request_clarification; increase_rationale_logging
governance: reject_promotion unless persistence or high-impact scope is present
Together, these signals may indicate runtime drift relevant to response quality, de-escalation, or safety posture. Stronger consequence requires persistence, corroboration, or policy-relevant convergence.
```

8. Relationship to Existing Stack Components

This bridge paper does not introduce a new subsystem. It maps existing weak-signal architecture into the AI safety monitoring domain.

Reflex-style salience supplies the short-horizon anomaly layer: it can raise attention without writing memory, updating baselines, or authorizing irreversible consequence.

Deviation governance supplies the structural event layer: it can compare runtime signals against scoped baselines, form events under persistence and hysteresis, attach rationale, and route outcomes into governed review or restriction.

Memory stratification supplies persistence discipline: not every salient runtime anomaly deserves durable storage, baseline mutation, or identity-layer update.

Experience-entry and memory-boundary contracts supply write discipline: ambiguous or contradiction-bearing events may be admitted, labeled, held, or quarantined without becoming canonical state.

Runtime governance supplies consequence discipline: capability, memory, review, and action consequences require declared predicates and auditable transition records.

Mapping: reflex salience → short-horizon flags; deviation governance → event store and predicates; memory stratification → write contracts; runtime governance → decision API.

9. Non-Goals and Scope Limits

This paper is deliberately limited.

It is not a general AI safety solution. It is not a full alignment paper. It is not a guarantee of safe AGI. It is not a formal verification framework. It is not a benchmark claim. It is not a replacement for red-teaming, policy evaluation, secure deployment practice, interpretability, capability control, or human oversight.

It also does not claim that weak-signal monitoring alone can prevent catastrophic risk. Weak-signal monitoring is a runtime architectural layer, not a universal safety theorem.

The claim is narrower: this architecture reduces certain classes of operational risk by improving early attention, promotion control, and auditability. It is not a substitute for other safety controls.

10. Failure Modes and Mitigations

A weak-signal safety monitor can fail in predictable ways.

Over-promotion: every weak anomaly becomes a governed safety state, producing false positives, excessive refusal, review fatigue, and brittle operation. Mitigation: monitor false-

positive rate, require governance TTLs, and automatically de-escalate unresolved A4 states when support decays.

Under-promotion: weak anomalies are recorded but never allowed to widen attention, making the system decorative logging rather than safety monitoring. Mitigation: track time-to-first-governance-request for high-impact paths and audit cases that remain in A1 or A2 despite repeated recurrence.

Salience-consequence collapse: a salient runtime anomaly automatically becomes restriction, memory update, or shutdown logic. Mitigation: enforce the transition rule that A4 requires a `governance_record_id` and at least one matched predicate.

Baseline contamination: unstable anomaly periods are used to update ordinary baselines, allowing the system to normalize unsafe drift. Mitigation: freeze baseline updates for affected scopes while events remain in A2, A3, A4, or Q0.

Rationale loss: promotion and resolution are not replayable, preventing later review. Mitigation: require rationale hashes, source monitor ids, matched predicates, and timestamp windows in every governance record.

Monitor absolutism: monitor output is treated as truth rather than as a fallible bounded signal. Mitigation: preserve monitor source, confidence, diagnostics, and quarantine pathways for suspected artifacts.

Review overload: too many weak signals are routed to humans without triage. Mitigation: use TTLs, automatic decay, batching by scope, and A3-only review routing except for high-impact proximity.

Stale degraded mode: reversible degradation remains active after the triggering signal decays. Mitigation: require TTL expiration, deterministic re-entry checks, and explicit closure reasons.

These mitigations reinforce the same design principle: weak signals should influence attention first, while stronger consequence requires measurable predicates and governance.

11. Conclusion

AI safety monitoring is a natural domain for weak-signal interpretation because runtime instability often begins before it becomes obvious. A subtle contradiction, schema inconsistency, subsystem disagreement, policy-boundary wobble, memory-write pressure, tool-use anomaly, or stance drift may not justify immediate hard action on its own. But it may still justify heightened attention.

The architecture proposed here separates that process into bounded observation, provisional interpretation, short-horizon salience escalation, and governed consequence. That separation

allows weak anomaly signals to matter without instantly becoming truth, memory, permission, restriction, or shutdown.

Deliverable recap: a minimal operational contract—state ladder, typed event schema, transition table, measurable predicates, and a governance API—that runtime teams can implement and audit. A natural next step would be a one-page reference implementation or checklist for integration into an existing monitoring stack.